

cqs — Code Quality Studio CLI

18 stacks · 533 rules · offline · CI-ready · Node 18+

INSTALL FROM NPM (ANYWHERE)

```
npm install -g cqs-audit
```

npmjs.com/package/cqs-audit

✓ Verify: **cqs --version**

v2.6.0

1 • Install & Everyday Use

Install → verify → run your first audit

🌐 INSTALL FROM NPM (PUBLIC — ANY MACHINE)

```
# Install globally from npm registry
npm install -g cqs-audit
```

```
# Verify what you actually got
cqs --version
```

```
# Upgrade later
npm install -g cqs-audit@latest
```

```
# nvm users: globals are per Node version
node --version && cqs --version
```

📦 Package: **cqs-audit** on npmjs.com/package/cqs-audit

NEW IN 2.4.0 — ADOPT ON AN EXISTING SUITE

Baseline	A real suite starts with debt. <code>--baseline-write</code> accepts today, <code>--baseline</code> fails only on <i>new</i> findings — so it can gate CI from day one.
Only what changed	<code>--changed</code> audits just the files this branch touched (committed, staged, unstaged, untracked). <code>--since <ref></code> picks the base.
Report opens by default	No <code>--open</code> needed. Skipped automatically for <code>--output json</code> / <code>summary</code> and when piped, so CI stays clean. <code>--no-report</code> opts out.
Every stack is audited	<code>cqs .</code> used to pick one stack and silently ignore the rest. A repo with specs, frontend and Java code now reports all three.
Files routed by import	A <code>.ts</code> file importing <code>@playwright/test</code> gets Playwright rules whatever it is named — so page objects are no longer scored as plain TypeScript.
Unaudited files are reported	Anything with no matching stack is listed at the end instead of vanishing.
<code>cqs-mcp</code> is a real command	The MCP server ships as its own binary; <code>cqs mcp-config</code> prints the client config for you.

⚡ CORE COMMANDS

COMMAND	DESCRIPTION
<code>cqs .</code>	Analyse current folder — routes every stack it finds
<code>cqs ./tests/</code>	Analyse a specific folder
<code>cqs ./tests/ --stack playwright</code>	Force a specific stack
<code>cqs ./tests/ --severity critical</code>	Show only critical findings
<code>cqs ./tests/ --output json > report.json</code>	Save JSON report for CI or sharing
<code>cqs ./tests/ --output summary</code>	One-line status — great for CI logs
<code>cqs ./tests/ --open</code>	Force the report even when piped or in CI
<code>cqs ./tests/ --no-report</code>	Terminal output only — skip the browser report
<code>cqs ./tests/ --category reliability</code>	Filter to one category
<code>cqs --read-report report.json</code>	Print summary of a saved JSON report
<code>cqs --read-report report.json --open</code>	Open saved report as HTML in browser
<code>cqs --list-stacks</code>	Show all 18 available stacks + IDs
<code>cqs ./tests/ --baseline-write .cqs-baseline.json</code>	Record current findings as accepted
<code>cqs ./tests/ --baseline .cqs-</code>	Fail only on findings beyond the baseline

COMMAND	DESCRIPTION
<code>baseline.json</code>	
<code>cqs --changed</code>	Audit only files changed vs <code>origin/main</code>
<code>cqs --changed --since develop</code>	Choose the base ref to compare against
<code>cqs --version</code>	Print the installed version (also <code>-v</code>)
<code>cqs serve --open</code>	Start the local server and open the studio, already signed in
<code>cqs serve --port 4005</code>	Use another port when 4000 is taken
<code>cqs serve --allow-write</code>	Permit the agents to edit files — off by default
<code>cqs mcp-config cursor</code>	Print ready-to-paste MCP config (or <code>claude</code>)
<code>cqs --help</code>	Show help text and version number

ALL 18 STACKS (--STACK <ID>)

UI AUTOMATION

 playwright

 playwright_java

 playwright_python

 selenium_java

 selenium_csharp

 cypress

MOBILE / MODEL-BASED

 appium_java

 tosca_xml

API TESTING

 restassured

 karate

 pytest_api

 postman

APPLICATION CODE

 java_api

 java_frontend

 typescript

 ts_frontend

 python_api

 python_frontend

 AI SETUP — ANTHROPIC & OPENAI

ANTHROPIC (CLAUDE)

```
# 1. Key from console.anthropic.com
# 2. Persist it in your shell profile
echo 'export ANTHROPIC_API_KEY=sk-ant-xxx' >> ~/.zshrc
source ~/.zshrc
# 3. Run — key picked up automatically
cqs ./tests/ --ai anthropic
```

OPENAI (GPT)

```
# 1. Key from platform.openai.com
echo 'export OPENAI_API_KEY=sk-xxx' >> ~/.zshrc
source ~/.zshrc
# 2. Run (default model gpt-4o-mini)
cqs ./tests/ --ai openai
```

Set the key **once** — every later run reads it from the environment.

 AI SETUP — BEDROCK & VERTEX

AWS BEDROCK

```
echo 'export AWS_REGION=us-east-1' >> ~/.zshrc
echo 'export AWS_ACCESS_KEY_ID=AKIAxxx' >> ~/.zshrc
echo 'export AWS_SECRET_ACCESS_KEY=xxx' >> ~/.zshrc
source ~/.zshrc
cqs ./tests/ --ai bedrock
```

GOOGLE VERTEX AI

```
# Service-account JSON from the GCP console
echo 'export VERTEX_PROJECT=my-proj' >> ~/.zshrc
echo 'export VERTEX_LOCATION=us-central1' >> ~/.zshrc
echo 'export
GOOGLE_APPLICATION_CREDENTIALS=~/.sa.json' >> ~/.zshrc
source ~/.zshrc
cqs ./tests/ --ai vertex
```

INLINE (CI — no env var)

```
cqs ./tests/ --ai anthropic --api-key sk-ant-xxx
cqs ./tests/ --ai openai --api-key sk-xxx
cqs ./tests/ --ai bedrock --aws-region us-east-1 \
  --aws-access-key KEY --aws-secret-key SECRET
cqs ./tests/ --ai vertex --vertex-project proj \
  --vertex-location us-central1 --vertex-key-file
sa.json
```

 CORE AI COMMANDS (AFTER KEY IS SET)

```
# Audit with AI second-eye review
cqs ./tests/ --stack playwright --ai anthropic
cqs ./tests/ --stack playwright --ai openai
cqs ./tests/ --stack playwright --ai bedrock
cqs ./tests/ --stack playwright --ai vertex
```

```
# Review only the critical files
cqs ./tests/ --ai anthropic --severity critical
```

```
# AI review + open HTML report
cqs ./tests/ --ai anthropic --open
```

```
# AI review + save JSON
cqs ./tests/ --ai anthropic --output json >
r.json
```

PICK A STRONGER MODEL

```
cqs ./tests/ --ai anthropic --model claude-
sonnet-5
cqs ./tests/ --ai openai --model gpt-4o
```

AI review runs **after** the offline rules and only on files that already have critical or warning findings.

🔗 CI / CD INTEGRATION

ADOPTING ON AN EXISTING SUITE (recommended)

```
# once – accept today's debt, commit the
file
cqs ./tests/ --baseline-write .cqs-
baseline.json

# every PR – fails only if this branch
makes it worse
cqs --changed --baseline .cqs-baseline.json
```

GITHUB ACTIONS

```
- name: Audit test quality
  run: cqs --changed --baseline .cqs-
baseline.json

# or: cqs ./tests/ --output summary
(gates on ALL criticals)
```

GITLAB CI

```
quality-gate:
  script: cqs ./tests/ --output json >
r.json
  artifacts: { paths: [r.json] }
```

PRE-COMMIT HOOK

```
cqs . --severity critical --output summary
|| exit 1
```

🔍 ADVANCED CHECKS — AUDITING AGAINST A RULE BOOK

WHAT YOU WANT TO CHECK	COMMAND
No hard waits / deterministic waits	<code>cqs ./tests/ --category reliability</code>
Page Object separation, no shared state	<code>cqs ./tests/ --category structure</code>
Assertion quality (no bare truthy checks)	<code>cqs ./tests/ --category assertions</code>
Locator strategy	<code>cqs ./tests/ --category selectors</code>
Secrets, TLS, credentials	<code>cqs ./tests/ --category security</code>
baseURL, retries, tracing, <code>.only</code>	<code>cqs ./tests/ --category ci_config</code>
Duplicated tests and setup blocks	<code>cqs ./tests/</code> — reported as <code>DUP-*</code>
Only what this branch changed	<code>cqs --changed --baseline .cqs-baseline.json</code>

Team standards a built-in rule does not cover go in **cqs-rules.json** at the repo root — for example retries capped at 1, quarantined tests needing a ticket, or no staging URLs in tests. Validate one before committing it: ask your assistant “*would this rule catch anything in ./tests?*”

🎯 OUTPUT FORMATS & ADVANCED

pretty Colour terminal — scores, findings, bars (default)

json Machine-readable — pipe to file for CI artefacts

summary One-liner: score 90 · 5 critical · 12 warning

ADVANCED COMMANDS

```
# Filter by category
cqs ./tests/ --category reliability

# Multiple paths
cqs ./src/tests/ ./e2e/ --stack playwright

# Plain text (no ANSI – for log files)
cqs ./tests/ --no-color

# Save + open later
cqs ./tests/ --output json > r.json
cqs --read-report r.json --open
```

USING CQS WITH CURSOR IDE

OPTION 1 — Terminal inside Cursor

```
# Open terminal in Cursor (Ctrl+`), then
run normally
cqs ./tests/ --stack playwright --open
# HTML report opens in your default browser
```

OPTION 2 — Cursor Task (tasks.json)

```
// .vscode/tasks.json
"tasks": [{
  "label": "cqs audit",
  "type": "shell",
  "command": "cqs ${workspaceFolder}/tests/ --open",
  "group": "test"
}]
```

OPTION 3 — Cursor AI + cqs together

1. Run: `cqs ./tests/ --output json > r.json`
2. Open `r.json` in Cursor
3. Ask Cursor AI: "Fix the critical findings in this report"
4. Cursor reads the JSON + your test files and applies fixes

MCP SERVER — CQS AS AN AI TOOL

BUILD THE MCP SERVER

```
# From the cqs CLI source directory
cd code-quality-studio/cli
node build-mcp.mjs
# → produces dist/cqs-mcp.js
```

CONFIG — SAME JSON FOR BOTH CLIENTS

```
{
  "mcpServers": {
    "cqs": {
      "command": "node",
      "args": ["/abs/path/dist/cqs-mcp.js"]
    }
  }
}
```

Cursor → `.cursor/mcp.json`

Claude Code → `~/.claude/settings.json`

MCP TOOLS EXPOSED

`cqs_audit` — audit a path, returns findings
`cqs_list_stacks` — list all 18 supported stacks
`cqs_read_report` — summarise a JSON report file

Once registered, ask the AI: "Audit my tests folder" — the AI calls `cqs_audit` automatically.

CLAUDE CODE /CQS SKILL

INSTALL THE SKILL (one-time)

```
# Create the skill directory
mkdir -p ~/.claude/skills/cqs

# Copy SKILL.md from the cqs repo
cp cli/bin/cqs-skill.md
~/.claude/skills/cqs/SKILL.md
```

```
# Or download directly (if cqs-audit is
installed globally)
cqs --version # verify cqs is working
```

USE IN CLAUDE CODE SESSION

```
# In any Claude Code session, just type:
/cqs

# Claude will ask you for the path, then
run the audit
# and summarise findings directly in the
conversation
```

EXAMPLE PROMPTS WITH SKILL

```
"Audit my tests and fix all critical issues"
"Run cqs on ./e2e/ and summarise what's wrong"
"Check my Playwright tests for quality issues"
"Review ./tests/ with AI — use Anthropic"
```

SKILL FILE LOCATION

```
~/.claude/skills/cqs/SKILL.md
```

Claude Code auto-loads skills from `~/.claude/skills/` — no restart needed after adding SKILL.md.

 REMEDIATION AGENT

Fixes findings in place. Every fix is re-audited before it is kept.

PREVIEW FIRST

```
cqs remediate ./tests/ --ai anthropic \
  --dry-run
# prints the diff, writes nothing
```

APPLY

```
cqs remediate ./tests/ --ai anthropic

# apply on a branch and commit
cqs remediate ./tests/ --ai anthropic \
  --branch fix/cqs --commit
```

FLAGS

- `--severity` what to fix (default critical)
- `--max-files` cap per run (default 10)
- `--force` write even if git is dirty
- `--provider` alias for `--ai`

 PR REVIEW BOT

Audits the files a PR changed and posts one review with inline comments.

IN GITHUB ACTIONS — NO FLAGS

```
cqs pr-review --stack playwright \
  --threshold 80
# repo, PR and token come from the env
```

MANUALLY, AGAINST ANY PR

```
cqs pr-review --repo owner/name --pr 42 \
  --token ghp_xxx --dry-run
```

WORKFLOW PERMISSION

```
permissions:
  contents: read
  pull-requests: write
```

- `--threshold` exits non-zero below the score
- `--only-added` comment only on added lines
- `--max-comments` cap inline comments (30)

 WHAT STOPS A BAD FIX

Both agents re-audit before trusting AI output. A fix is **rejected** when it:

- introduces a new critical finding
- leaves the file unchanged
- returns no code block
- drops more than half the file

THE CLASSIC CHEAT

Deleting an assertion removes the finding but leaves a test that verifies nothing. Re-auditing catches it — the missing assertion is itself a critical.

GIT SAFETY

remediate refuses to write unless the tree is clean, so every applied fix is revertable with `git checkout`.

CQS SERVE — THE STUDIO ON YOUR MACHINE

One command starts a local server *and* the full web studio it serves. The page arrives already signed in — no token to copy, no second port.

START · STOP

```
# start it in the suite you want to scan
cd ~/my-project && cqs serve --open
# → Open http://127.0.0.1:4000

# stop: Ctrl-C. still busy? an older one is up:
pkill -f "cqs serve"
```

Omit `--open` for the API alone (SSH, `curl`). A second one on a taken port exits with `EADDRINUSE` instead of stealing it — so a stale-looking banner means you are talking to the *old* one. `--port 4005` runs two at once.

WHICH PATHS CAN IT SCAN?

`./tests` Relative to **where you started serve** — not the repo, not the browser. Wrong folder → **Path not found**, which prints the folder it resolved from.

`/abs/path` Any directory on the machine. No confinement to the start folder, so one server can audit several repos.

`~/tests` Will not work — your shell expands `~`, the API never sees a shell. Type the full path.

Safe by default: bound to 127.0.0.1 only · random token per run on every request except `/health` · origin allowlist · cannot write without `--allow-write`.

OFFLINE AI — THE CQS-AI ADD-ON

A ~104 MB classifier that runs on your machine. **No API key, no network.** A separate package on purpose — if it fails to install, `cqs` is unaffected.

FROM THE STUDIO (EASIEST)

```
cqs serve --open
# Files → Add-ons → Local AI → Install
# then ▶ Run on N files
```

Installs to `~/cqs/addons` — no sudo. **Stop** frees its ~500 MB; **Remove** uninstalls it.

OR THE TERMINAL

```
npm install -g cqs-ai
cqs-ai ./tests/login.spec.ts
cqs-ai warm # fetch model now
cqs-ai serve # port 4100
```

It only runs where the rules found nothing. A hard wait in plain sight should be reported by a rule, not guessed at — so a suite full of them reports "*skipped, fix those first*". What it judges: fragile selectors, test scope, page-object adherence, assertion strength — each a label and a probability, reported above 65%. Never prose.

cqs v2.6.0 · **npm install -g cqs-audit** · AI: anthropic | openai | bedrock | vertex · MCP: cqs_audit | cqs_report | cqs_list_stacks | cqs_list_rules | cqs_validate_rules | cqs_test_rule | cqs_read_report · Skill: /cqs
· Offline AI: **npm i -g cqs-ai** · Agents: remediate | pr-review · Packaging → **cqs-admin-cheatsheet.pdf**